

Tour-Planner

Protocol

App Architecture

Layers

Following the requirements of applying a layer-based architecture, there are three backend layers with the additional test project and the Angular Frontend. The class diagrams for each controller, service and repository can be found on the next page.

For more information, refer to the “Unit Test” Section.

We split our Angular Application into 5 folders.

The “components” folder has reusable web-UI components that we make use of across different sections of our pages.

The “enums” folder contains any enums that we need, though it currently only has the `TransportType`, which matches the defined enum in the backend.

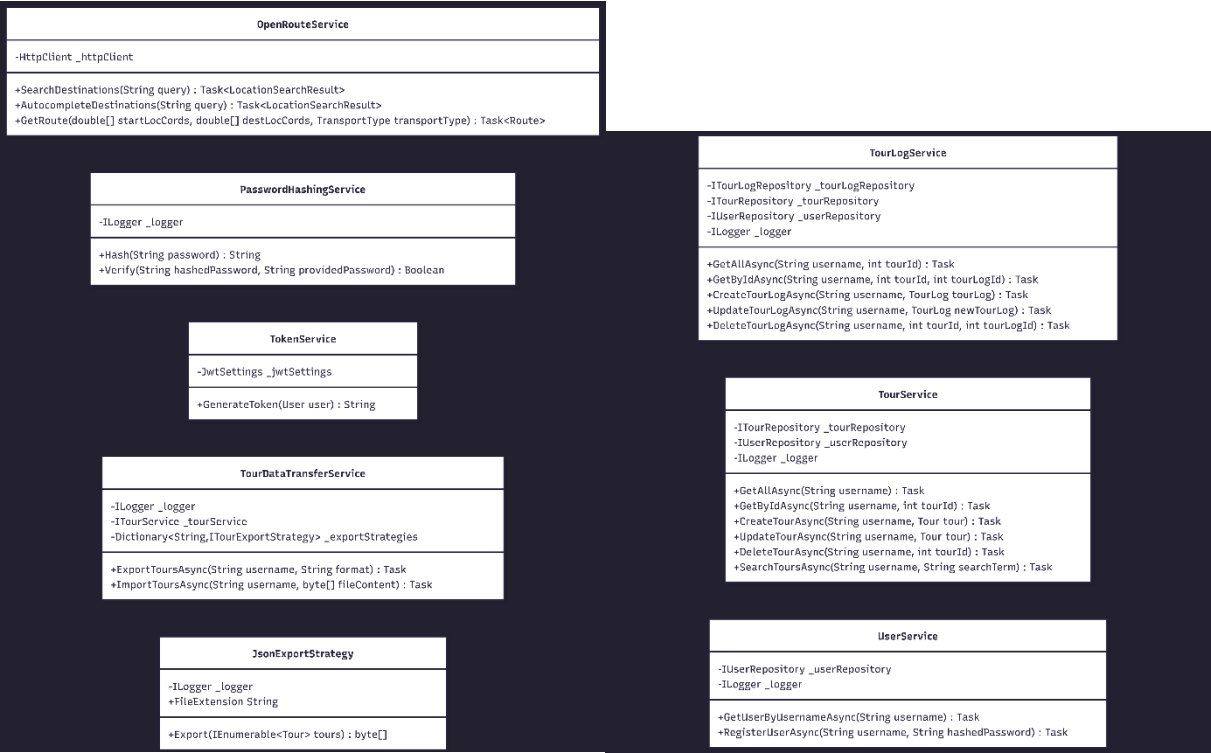
The “interfaces” folder contains interfaces for different models of the backend that are to be used in the frontend. Each file (named “[name].d.ts”) has the interface to be used for the frontend and an interface to be used when communicating with the backend.

The “pages” folder contains all the different pages of our site that can be visited. They are all mapped to a URL in `app.routes.ts`.

The “services” folder contains services that can be injected in the pages or components and usually serve as the connection to the backend.



The Presentation Layer (TourPlanner.Api)



The Business Layer (TourPlanner.BLL)

TourRepository

-TourPlannerDbContext _context

+TourRepository(TourPlannerDbContext context)
+GetAllByUserIdAsync(int userId) : List<Tour>
+GetByIdAsync(int tourId) : Tour?
+AddAsync(Tour tour) : Task
+DeleteAsync(int tourId) : Task
+UpdateAsync(Tour tour) : Task
+SearchAsync(int userId, String query) : List<Tour>

TourLogRepository

-TourPlannerDbContext _context

+TourLogRepository(TourPlannerDbContext context)
+GetAllByTourIdAsync(int tourId) : List<TourLog>
+GetByIdAsync(int tourLogId) : TourLog?
+AddAsync(TourLog log) : Task
+DeleteAsync(int tourLogId) : Task
+UpdateAsync(TourLog log) : Task

UserRepository

-TourPlannerDbContext _context

+UserRepository(TourPlannerDbContext context)
+GetUserByUsernameAsync(String username) : Task
+InsertUserAsync(User user) : Task

The Data Access Layer (TourPlanner.Dal)

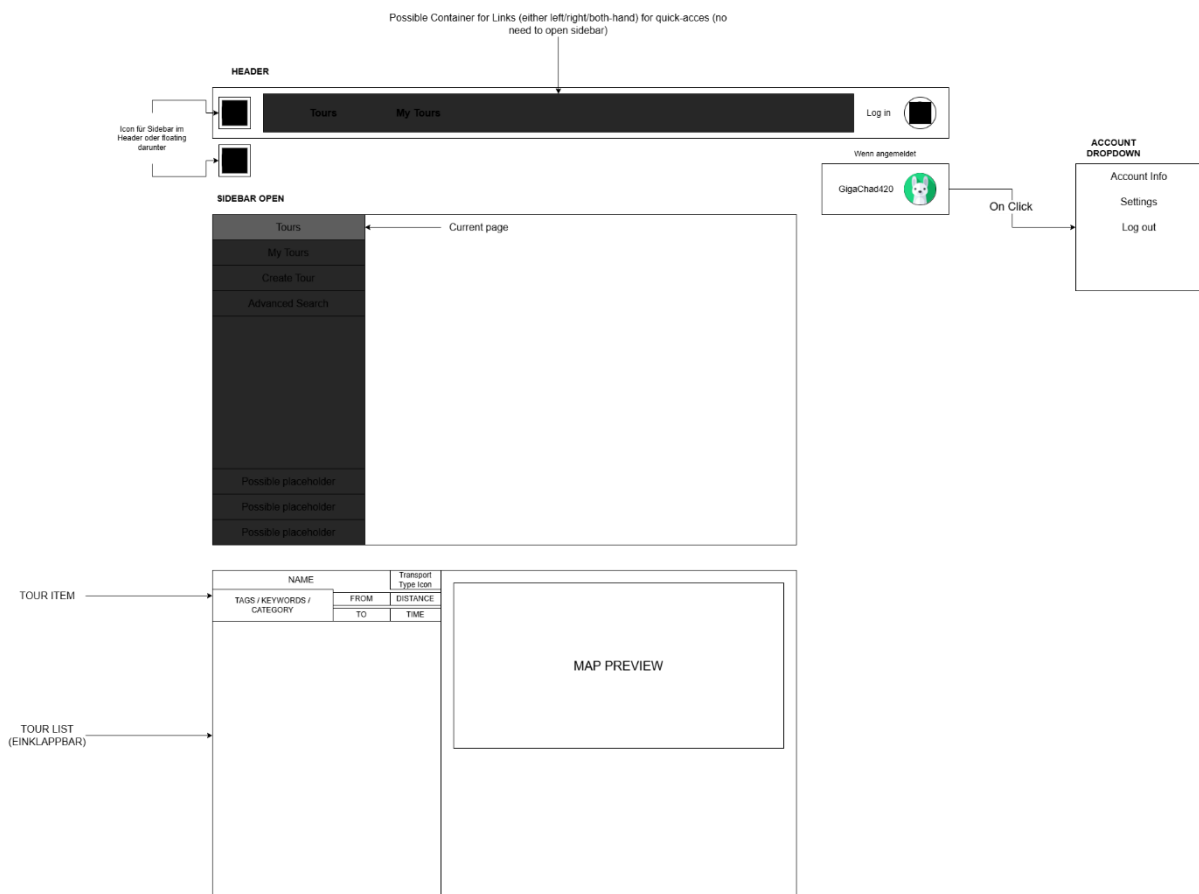
JWT-Token

We decided to send the token to the frontend as an HTTP-only cookie because it is less vulnerable to XSS attacks. As the frontend therefore does not have access to this cookie, the token also gets sent back in the request cookie as opposed to the authorization header.

UX

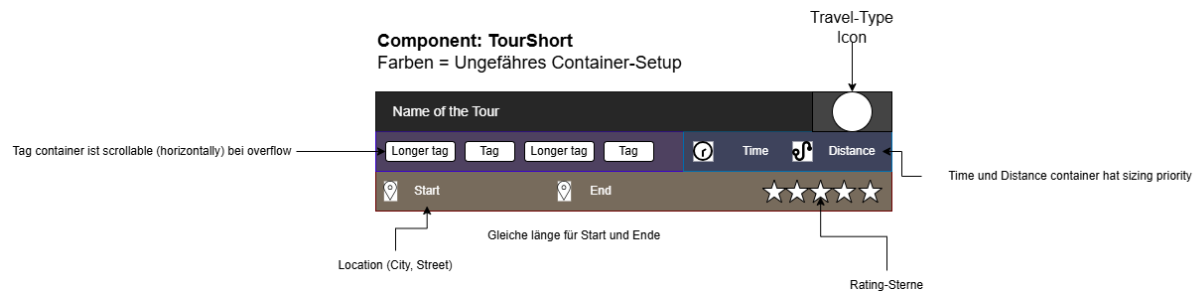
Wireframes

A header with a navbar including links to the most used pages is provided at the top. On the right, either a log in button or the account dropdown component is shown, which would then lead to further account related pages.



1 Wire_mogged.drawio

A sidebar opens on the left-hand side on the tours page, which is a list of all the tours one can see.



Example

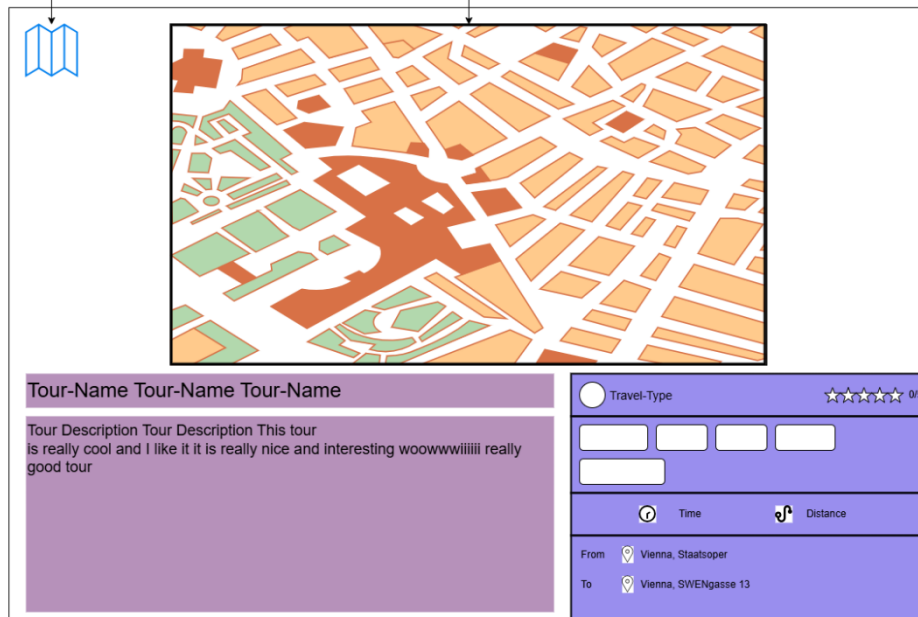
2 Tour_Short_Iter2.drawio

On selecting one of those tours, the full tour can be seen on the right side. (Explanation of the individual elements)

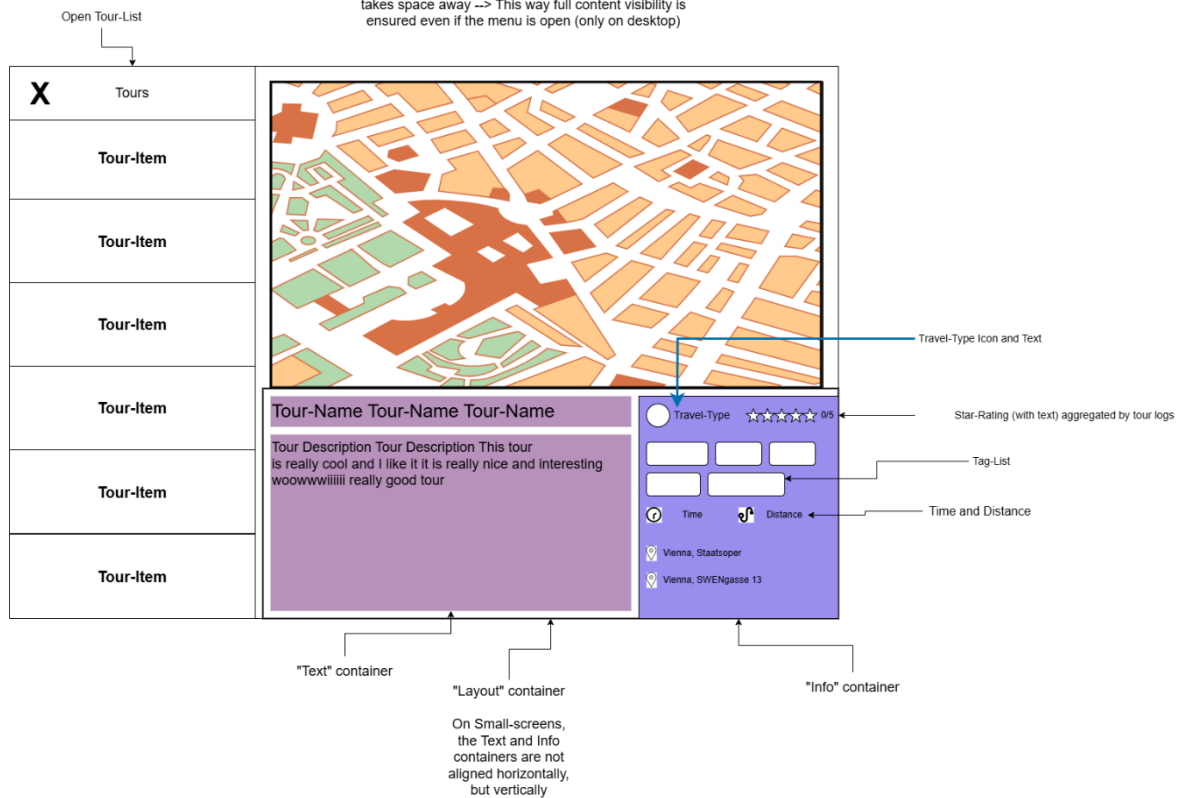
Button for List

Map-Widget (And page in general) with flexible width

This Wireframe does not include the header on the top



An open TourList does not overlap over the page, it takes space away --> This way full content visibility is ensured even if the menu is open (only on desktop)





3 Tour_Full_Iter2.drawio

The create and edit tour pages orient their layouts similar to how the full tour view is set up.

Create/Edit Tour

Tour Name

Tour Description

Transport Type Icon

Transport Type

Tags

Starting Location

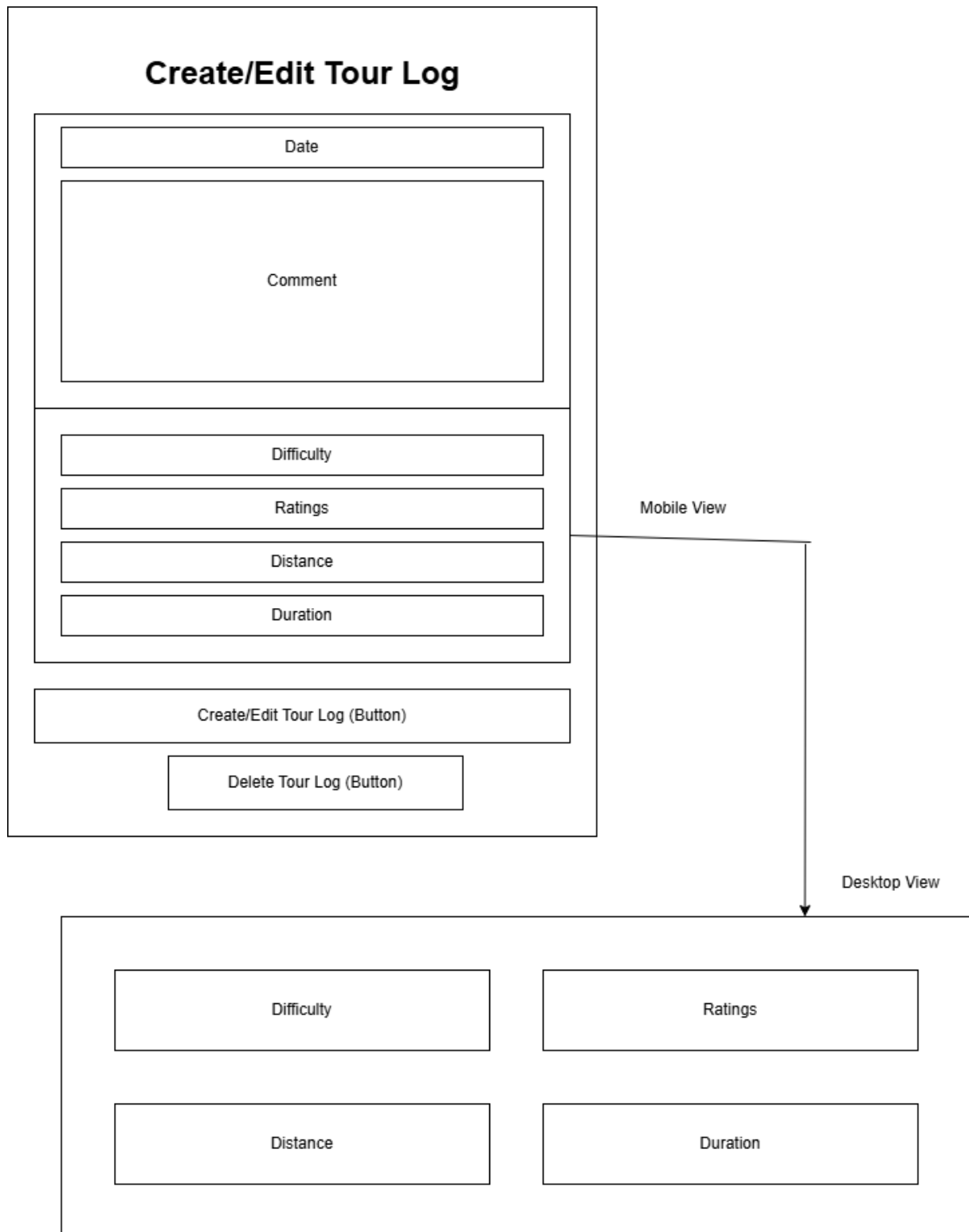
Ending Location

Create/Edit Tour (Button)

Delete Tour (Button)

4 Tour_Create_or_Edit_Iter1.drawio

The create and edit tour log pages are based on a similar layout to the create and edit tour pages for the user to be familiar with the elements.



5 Tour_Log_Create_or_Edit_Iter1.drawio

Library Decisions

We used `Microsoft.Extensions.Logging.Log4Net.AspNetCore`, which essentially allows us to use Log4Net while writing the logging implementation in our code as if we used the `Microsoft.Extensions` “ILogger”. We decided to use this, because we already wrote our

logging using the ILogger. That way, we could still make use of Log4Net without having to change the implementation in our code.

Lessons Learned

Not well typed APIs are hard to learn and to work with.

Time management is absolutely essential for a project.

Planning can cut down on time wasted in refactoring and adjusting.

Implemented Design Patterns

One of the patterns we used is the Façade Pattern. With Leaflet, there are a lot of individual parts like “L.Map()”, “L.tileLayer()” or “L.marker()”. Instead of setting these values individually, we can pass it to the Façade Service, which sets these for us while we only have to call only one function that the Façade provides like “initMap()”.

Another design pattern, which we used is the Strategy Pattern. We initially had the idea of allowing for different file formats for the tour data that we export. Though these work differently under the hood, they essentially still just put the information into a file that can be exported. To group them all into a single functionality that is independent of the individual implementation, the strategy provides a way to do the exporting of data as different formats.

Unit Testing Decisions

There are no unit tests for the controllers in the API layer. This is because most of the controllers merely receive a request, pass the parameters to the services in the Business Logic Layer, then return a response. As such, Unit tests would test the ASP.NET Core Framework and not any Implementation specifically made by us.

There are no unit tests for the repositories in the Data Access Layer. This is because the repositories communicate with the PostgreSQL database using the OR-Mapper. As such, unit tests would test the Entity Framework and not any Implementation specifically made by us.

The unit tests concentrate on the Business Logic Layer with each service having their own set of unit tests.

There are five services in total that require unit tests for their respective methods:

- UserService
 - GetUserByUsernameAsync(string username): Task<User>
 - RegisterUserAsync(string username, string hashedPassword): Task

- TourService
 - GetAllAsync(string username): Task<List<Tour>>
 - GetByIdAsync(string username, int tourId): Task<Tour?>
 - CreateTourAsync(string username, Tour tour): Task
 - UpdateTourAsync(string username, Tour tour): Task<bool>
 - DeleteTourAsync(string username, int tourId): Task<bool>
 - SearchToursAsync(string username, string? searchTerm): Task<List<Tour>>
- TourLogService
 - GetAllAsync(string username, int tourId): Task<List<TourLog>>
 - GetByIdAsync(string username, int tourId, int tourLogId): Task<TourLog?>
 - CreateTourLogAsync(string username, TourLog tourLog): Task
 - UpdateTourLogAsync(string username, TourLog tourLog): Task<bool>
 - DeleteTourLogAsync(string username, int tourId, int tourLogId): Task<bool>
- TokenService
 - GenerateToken(User user): string
- PasswordHashingService
 - Hash(string password): string
 - Verify(string hashedPassword, string providedPassword): bool

User Service Tests

The User Service is called by the Authentication Controller. Therefore, it is vital that this service works. If this service does not work, then no User can use the application because all endpoints beside the Authentication Controller require a token set by the backend in an HTTP only cookie.

Both functions require a username that then determines whether the User with that specific username exists. As such, each function has two tests, one where the user exists and the other where the user does not exist.

For GetUserByUsernameAsync(), the user should already exist. One unit test ensures that the user is returned if it does exist. Otherwise, the other unit test ensures that an exception is thrown indicating that the user could not be found.

For RegisterUserAsync(), the user should not exist yet. One unit test ensures that the user data is inserted into the database if there is no existing instance of the user yet. Otherwise, the other unit test ensures that an exception is thrown indicating that the user already exists in the database.

Tour Service Tests

The Tour Service handles everything related to tours. The most critical part of this service is ensuring that no user may access tours that do not belong to them. If the service does

not protect tours from users to whom they do not belong, then the data of one's own tour may become corrupted, potentially rendering them useless to their owner.

Tour Log Service Tests

The Tour Log Service handles everything related to tour logs. Similar to the Tour Service, the most critical part is ensuring that no user may access tour logs that do not belong to them. In addition, there is an added risk of the tour log service assigning the tour log to a different tour.

Token Service Tests

The Token Service generates a JSON Web Token (JWT) that identifies the user after login. This token is required in the majority of endpoints such as to access the tour and tour log altering methods as well as for logging in via the “me” endpoint. Therefore, this class is significantly important for the application, despite only having one method for generating tokens. The tests ensure that the token is not malformed.

Password Hashing Service Tests

The Password Hashing Service both hashes plain-text passwords and verifies them with existing hashes. This service is especially crucial because passwords could be guessed more easily or verified incorrectly. In these cases, such issues would compromise every user account registered with this application.

Unique Feature

We decided to add tags to our tours. These tags help with searching for tours and to provide additional context using concise words.

Tracked Time

UI design and wireframes: about 6-7 hours

Angular project setup: about 1 hours

User authentication (registration and login): about 8 hours (includes backend)

Database design schema creation: about 1.5 hours

Backend architecture planning and setup (BL, DAL, Controllers) about 3 hours

Tour CRUD functionality: about 3 hours

Tour Log CRUD functionality: about 3 hours

User input validation: about 1 hour

PostgreSQL & ORM integration: about 4 hours

OpenRouteService API integration: about 10 hours (difficulties with API documentation)

Leaflet map integration: about 3 hours (difficulties with API documentation)

Import and Export feature: about 4 hours (debate about which file format and possible support for multiple file formats)

Logging implementation: about 2 hours (found an extension that lets us use ILogger while also using Log4Net)

Unit Testing: about 10 hours (difficulties with thinking in theories rather than individual tests)

Documentation: about 10 hours (broken down into multiple sessions, so might be more or less)

Link to GIT

<https://github.com/emubi50/tour-planner.git>